Tokenizer => matnni modelning alifbosiga aylantiradi; dataset builder esa shu alifbodagi matnni o'qitishga tayyor binar fillarga joylaydi

BPE character va word darajasi o'rtasidagi muvozanatni beradi. u ko'p uchraydigan so'zlarni bitta token qiladi.

=> Tokenizer matnni doim bir xil kodlashi uchun merge rules aynan o'rganilgan tartibda qo'llanilishi kerak.

=> Trainda BPE eng ko'p uchraydigan juftlikni birlashtiradi va har bir birlashmani | merges.txt | ga ketma ket yozadi.

=> Vocab hajmi model hajmi bilan sequence uzunligi o'rtasidagi kelishuvdir.
Vocab oshsa, embedding jadvali kattalashadi, lekin matn kamroq tokenga bo'linadi.

Kichik vocab - embedding kichik, har bir token ko'p marta ko'riladi. va yaxshi o'rganiladi. Lekin mant ko'proq tokenga bo'linadi. compute oshadi, context windowga kamroq matn sig'adi.

Katta vocab - matn ziqroq, lekin embedding kattalashadi. va kichik korpusda ko'p tokenlar juda kam uchraydi. Bunday tokenlarning vektorlari yaxshi o'rganilmaydi.

<eos> Hujjat tugaganini bildiradi.
<bos> Hujjat boshlanishini belgilaydi
<pad> Qisqa ketma ketliklarni bir xil uzunlikka to'ldiradi.
<unk> Noma'lum belgi.

Modelga matn chegaralari yoki maxsus holatlarni tushuntirish uchun ishlatiladigan maxsus xizmatchi tokenlar.

Round-trip test

Matnni encode qilib yana qayta decode qilganda asl matnni olish kabilarini tekshirish.

Document level data splitting

Train va validation hujjat darajasida ajratiladi, chunki bitta hujjatning gaplari ikkala to'plamga tushsa, validation sun'iy ravishda past chiqadi. Overfitting ham oldi olinadi.

Nega gap darajasida emas?

→ Bitta maqolaning birinchi yarmi train'ga ikkinchi yarmi validation'ga tushsa, model validation'da mavzu, muallif va uslubni allaqachon biladi. Natijada model haqiqatdagidan yaxshiroq ko'rinadi.